

```
1 #%%
2 from datetime import datetime, timedelta
3 from statsmodels.tsa.seasonal import
  seasonal_decompose
4 import numpy as np
5 import pandas as pd
6 import warnings
7 warnings.filterwarnings("ignore")
8 #%% md
9 ##### Importing the dataset comprising data of
  children with illegal immigration status or no parent
  or guardian
10 #%%
11 df = pd.read_csv("
  HHS_Unaccompanied_Alien_Children_Program.csv")
12 df.head()
13 #%% md
14 ##### Checking for null values and duplicates
15 #%%
16 df.isna().sum()
17 #%%
18 df.duplicated().sum()
19 #%% md
20 ##### Converting data to pandas date-time format
21 #%%
22 df["Date"] = pd.to_datetime(df["Date"])
23 df.head()
24 #%% md
25 ##### Arrange dates in ascending order
26 #%%
27 df.sort_values(by=["Date"], inplace=True)
28 df.reset_index(drop=True, inplace=True)
29 df.head()
30 #%% md
31 ##### Retrieving weekday names for each date in the
  dataset
32 #%%
33 df["Day of Week"] = list(map(lambda x: x.strftime("%A
  "), df["Date"]))
34 df = df[["Date", "Day of Week", "Children apprehended
  and placed in CBP custody*", "Children in CBP custody"]
```

```

34 , "Children transferred out of CBP custody", "Children
    in HHS Care", "Children discharged from HHS Care"]]
35 df.head()
36 #%% md
37 ##### Finding out the total number of records on each
    day of week
38 #%%
39 weekday_number = df["Day of Week"].value_counts().
    reset_index()
40 weekday_number["Day of Week Number"] = weekday_number
    ["Day of Week"].map({
41     "Sunday": "1",
42     "Monday": "2",
43     "Tuesday": "3",
44     "Wednesday": "4",
45     "Thursday": "5",
46     "Friday": "6",
47     "Saturday": "7"
48 })
49 weekday_number = weekday_number.sort_values(by=["Day
    of Week Number"]).reset_index(drop=True)
50 weekday_number
51 #%% md
52 Looking at the numbers above, we can see that the
    number of days on which children were recorded is
    relatively consistent from Sunday through Thursday.
    From Monday through Thursday, the figures range
    between 145 and 150 days. Sunday is somewhat lower,
    at 130 days, but it is still reasonably close to this
    range.
53
54 Friday, however, stands out as an exceptionally low
    number of recorded days. Data is recorded for only
    two Fridays, compared with more than 130 recorded
    days for each of the other weekdays. This substantial
    difference suggests that Friday may represent a
    distinct pattern in the data and warrants further
    investigation.
55
56 There is one assumption that we can make here, which
    is deleting those two Friday records and assuming

```

56 that services are not provided on Fridays and Saturdays. But in reality, these services are provided 24 hours a day and 7 days a week. So we need to include these days on which data is not recorded and then somehow deal with the null values for number of children on these days.

57 `### md`

58 `#### Creating a new dataset with dates on which data is not recorded`

59 `###`

60 `df_2 = pd.DataFrame(columns=df.columns)`

61 `df_2["Date"] = np.setdiff1d(pd.date_range(df["Date"].min(),df["Date"].max()).values,df["Date"].values)`

62 `df_2["Day of Week"] = list(map(lambda x: x.strftime("%A"), df_2["Date"]))`

63 `df_2[['Children apprehended and placed in CBP custody *', 'Children in CBP custody', 'Children transferred out of CBP custody', 'Children in HHS Care', 'Children discharged from HHS Care']] = np.nan`

64 `df_2.head()`

65 `### md`

66 `#### Concatenating the original dataset with the new dataset comprising unrecorded dates`

67 `###`

68 `df_3 = pd.concat([df,df_2]).sort_values(by=["Date"]).reset_index(drop=True)`

69 `df_3.drop(columns=["Day of Week"], inplace=True)`

70 `df_3.set_index("Date", inplace=True)`

71 `df_3.head()`

72 `### md`

73 `#### Checking the data types of each feature`

74 `###`

75 `df_3.dtypes`

76 `### md`

77 As we observe the above data types, the feature * Children in HHS care* is of object type. The other features are of float type. In reality, these features denote number of children going through their respective stage. This means that such features ought to be integers.

78

79 All the features except the *Children in HHS Care* are floating points, which makes it unnecessary for us to convert into integers, as they are of numeric type. The values for such features end with ".0". The *Children in HHS Care* is of object type, which is a non-numeric type - this forces us to convert it to numeric type. Before type conversion, let us see how the values of *Children in HHS Care* look like:

80 #%%

81 print(df_3["Children in HHS Care"].unique().tolist())

82 #%% md

83 As we observe, there are commas in each element of *Children in HHS Care* column. We need to remove those commas and convert the data type into floating point so that the data type matches with that of other columns.

84 #%%

85 df_4 = df_3.copy()

86

87 df_4["Children in HHS Care"] = list(map(lambda x: "".join(x.split(",")), df_4["Children in HHS Care"]))

88 #%% md

89 From the above error, we can say that there are some floating point values for the column *Children in HHS Care*. We need to explicitly convert all values to string, then remove those commas and then convert them to floating point.

90 #%%

91 df_4["Children in HHS Care"] = list(map(lambda x: "".join(str(x).split(",")), df_4["Children in HHS Care"]))

92 df_4["Children in HHS Care"] = df_4["Children in HHS Care"].astype("float64")

93 df_4.head()

94 #%% md

95 Now, let us make use of data interpolation, which figures out values for each null value by itself based on non-null values around each of them.

96 #%%

97 df_4 = df_4.interpolate().copy()

```

98 df_4.head()
99 #%% md
100 As we observe the above dataset, the values are in
    decimal points. In reality the features denote
    number of children in each phase - this means that
    the elements should also be integers. So, we shall
    round all the elements to their nearest integers.
101 #%%
102 df_5 = df_4.round(0)
103 df_5.head()
104 #%% md
105 Now, we need to have a look for patterns for each
    column. The best way is to visualise the data.
106 #%%
107 import matplotlib.pyplot as plt
108 import seaborn as sns
109 #%%
110 df_6 = df_5.reset_index()
111 df_6.head()
112 #%% md
113 Let us export the data to a CSV file. We shall
    perform data visualisation in Tableau.
114 #%%
115 df_6.to_csv("HHS_Program_Cleaned.csv")
116 #%% md
117 ![image.png](attachment:8349c1fb-5268-44f9-b861-
    57302017ca4f.png)
118 #%% md
119 ![image.png](attachment:75c8d246-b484-45fe-9503-
    9eff84cc9989.png)
120 #%% md
121 ![image.png](attachment:b02de321-980f-4c2b-a380-
    df84bbf275ac.png)
122 #%% md
123 ![image.png](attachment:532435be-a0f1-4e5b-af60-
    80611a4e5d9c.png)
124 #%% md
125 From the above graphs, we can infer that Children
    who were considered by CBP, those already going
    through CBP custody and those done with the CBP
    custody and transferred to HHS care follow a very

```

125 similar pattern. Same is the case with the children
in HHS care and those who got discharged from it.

126

127 There is another important aspect extracted from the
dataset. The number of children undergoing each
phase of the program has gone drastically down in
February 2025, and the same level is maintained
throughout the rest of the time. This is mainly due
to decrease in illegal immigration to the United
States, thanks to harsh restrictions set up by the
Trump government. Trump came into power on January
20, 2025. Some of his initial executive orders
include crackdown of illegal immigration - this is
the main driver behind numbers falling down.

128 `### md`

129 `#### Detecting Outliers using Interquartile Range
Method`

130 `###`

131 `df_temp = df_6.drop(columns="Date")`

132

133 `Q1 = df_temp.quantile(0.25) # Lower Quartile`

134 `Q3 = df_temp.quantile(0.75) # Upper Quartile`

135

136 `IQR = Q3 - Q1`

137

138 `lower_limit = Q1 - 1.5 * IQR`

139 `upper_limit = Q3 + 1.5 * IQR`

140

141 `df_temp[(df_temp < lower_limit) | (df_temp >
upper_limit)].sum()`

142 `### md`

143 As we observe, there are no values below the lower
limit and above the upper limit. This means that
there are no outliers at all in our dataset.

144 `###`

145 `df_6.head()`

146 `### md`

147 `#### Renaming all columns to simple names`

148 `###`

149 `df_7 = df_6.copy()`

150 `df_7.rename(columns={`

```

151     "Date" : "Date",
152     "Children apprehended and placed in CBP custody
    *" : "Placed_In_CBP",
153     "Children in CBP custody" : "CBP_Custody",
154     "Children transferred out of CBP custody" : "
    CBP_To_HHS",
155     "Children in HHS Care" : "HHS_Care",
156     "Children discharged from HHS Care": "
    Discharged_From_HHS"
157 },inplace=True)
158
159 df_7.head()


---


160 ### md
161 ##### Creating lags and rolling averages for each
    column


---


162 ###
163 # Retrieving all parameters of children
164 list_of_columns = df_7.columns.values.tolist()[1:]
165
166 # New Lag Columns
167 lag_by_1 = [i+"_lag_1" for i in list_of_columns]
168 lag_by_7 = [i+"_lag_7" for i in list_of_columns]
169 lag_by_14 = [i+"_lag_14" for i in list_of_columns]
170
171 # Columns for Rolling averages
172 rolling_avg_7 = [i+"_rolling_avg_7" for i in
    list_of_columns]
173 rolling_avg_14 = [i+"_rolling_avg_14" for i in
    list_of_columns]
174
175 # Calculating lag features
176 df_7[lag_by_1] = df_7[list_of_columns].shift(1)
177 df_7[lag_by_7] = df_7[list_of_columns].shift(7)
178 df_7[lag_by_14] = df_7[list_of_columns].shift(14)
179
180 # Calculating rolling averages
181 df_7[rolling_avg_7] = df_7[list_of_columns].rolling(
    7).mean()
182 df_7[rolling_avg_14] = df_7[list_of_columns].rolling
    (14).mean()


---


183 ###

```

```

184 df_7.head(5)
185 #%% md
186 ##### Defining Flow based signals
187 #%%
188 # Defining the net flow of children right from
    entering HHS care to discharging them
189 df_7["Net_HHS_Flow"] = df_7["CBP_To_HHS"] - df_7["
    Discharged_From_HHS"]
190
191 # Rolling averages of Net HHS
192 df_7["Net_HHS_Flow_rolling_avg_7"] = df_7["
    Net_HHS_Flow"].rolling(7).mean()
193 df_7["Net_HHS_Flow_rolling_avg_14"] = df_7["
    Net_HHS_Flow"].rolling(14).mean()
194
195 # Rolling standard deviation across 7 days
196 df_7["Net_HHS_Flow_rolling_std_7"] = df_7["
    Net_HHS_Flow"].rolling(7).std()
197
198
199 df_7.head()
200 #%% md
201 Flow acceleration is just an additional and a useful
    feature, but not mandatory. It is just to determine
    how inflow pressure changes as days pass on
202 #%% md
203 ##### Adding calender effects
204 #%%
205 # Day of Week
206 df_7["Day_Of_Week"] = df_7["Date"].dt.dayofweek
207
208 # Month
209 df_7["Month"] = df_7["Date"].dt.month
210
211 # If the day is weekend or not
212 df_7["Is_Weekend"] = (df_7["Day_Of_Week"] >= 5).
    astype("int64")
213
214 df_7.head(8)
215 #%% md
216 ##### Time-Series Decomposition

```



```
217 #%%
218 decomp = seasonal_decompose(
219     x = df_7["HHS_Care"],
220     model = "additive",
221     period = 7
222 )
223 #%%
224 decomp_df = pd.DataFrame(columns=["Date", "Observed",
225     "Seasonal", "Trend", "Residual"])
226 decomp_df["Date"] = df_7["Date"].copy()
227 decomp_df["Observed"] = decomp.observed
228 decomp_df["Seasonal"] = decomp.seasonal
229 decomp_df["Residual"] = decomp.resid
230 decomp_df["Trend"] = decomp.trend
231 decomp_df.head()
232 #%%
233 decomp_df.to_csv("HHS_decomposition.csv")
234 #%% md
235 ![image.png](attachment:50639733-2b7a-49bb-8b5e-40f5084117e6.png)
236 #%%
237 df_7.head()
238 #%%
239
240 #%%
241
242 #%%
243
```